

Curriculum (Advanced)

```
/*
 * configurator_v2.c
 *
 * A user-specifiable C generative intelligence fabric.
 *
 * Internal C identifiers intentionally preserve the xN object naming style used
 * by configurator.c. Runtime object names, input names, flow names, and output
 * names are user-specified through CLI flags, config files, or the micro UI.
 *
 * Build:
 *     cc -std=c11 -Wall -Wextra -pedantic -O2 configurator_v2.c -o
configurator_v2
 *
 * Examples:
 *     ./configurator_v2 --object-name x33 --input-name x1 --input-value 01 \
 *         --input-width 3 --flow-name x15 --flow-type cartesian \
 *         --output-name x31 --output-target x33.txt
 *
 *     ./configurator_v2 --sample-config x33.fabric
 *     ./configurator_v2 --config x33.fabric
 *     ./configurator_v2 --ui
 *
 * Config format:
 *     object_name=x33
 *     input_name=x1
 *     input_value=0123456789
 *     input_width=7
 *     flow_name=x15
 *     flow_type=cartesian
 *     output_name=x31
 *     output_target=x33.txt
 *     separator=\n
 *     end
 *
 * Supported flow_type values:
 *     cartesian - generate every fixed-width combination from input_value
 *     literal   - write input_value once
 *     repeat    - write input_value input_width times
 *     reverse   - write input_value reversed once
 */

#include <stdio.h>
#include <stdint.h>
#include <stdbool.h>
#include <limits.h>
#include <string.h>
#include <stdlib.h>
#include <ctype.h>
```

```

#define x0 64
#define x1 512
#define x2 32
#define x200 4096

typedef struct {
    void* x4;
    bool (*x5)(void* x4, const char* x6, uint64_t x7);
    bool (*x8)(void* x4, char x9);
    bool x10;
} x3;

typedef enum {
    x11 = 0,
    x12 = 1,
    x13 = 2,
    x14 = 3
} x15;

typedef struct {
    bool x16;
    char x17[x0];
    char x18[x0];
    char x19[x1];
    uint64_t x20;
    char x21[x0];
    x15 x22;
    char x23[x0];
    char x24[x1];
    char x25[x0];
    char x26[x1];
    char x27[x1];
} x28;

typedef struct {
    x28 x29[x2];
    uint64_t x30;
} x31;

static bool x32(char* x33, uint64_t x34, const char* x35) {
    size_t x36;

    if (x33 == NULL || x35 == NULL || x34 == 0) {
        return false;
    }

    x36 = strlen(x35);
    if (x36 >= x34) {
        return false;
    }

    memcpy(x33, x35, x36 + 1);
    return true;
}

```

```

}

static int x37(int x38) {
    return tolower((unsigned char)x38);
}

static bool x39(const char* x40, const char* x41) {
    if (x40 == NULL || x41 == NULL) {
        return false;
    }

    while (*x40 && *x41) {
        if (x37(*x40) != x37(*x41)) {
            return false;
        }
        ++x40;
        ++x41;
    }

    return *x40 == '\\0' && *x41 == '\\0';
}

static char* x42(char* x43) {
    char* x44;
    size_t x45;

    if (x43 == NULL) {
        return NULL;
    }

    while (*x43 && isspace((unsigned char)*x43)) {
        ++x43;
    }

    x45 = strlen(x43);
    while (x45 > 0 && isspace((unsigned char)x43[x45 - 1])) {
        x43[x45 - 1] = '\\0';
        --x45;
    }

    x44 = strchr(x43, '#');
    if (x44 != NULL) {
        *x44 = '\\0';
        x45 = strlen(x43);
        while (x45 > 0 && isspace((unsigned char)x43[x45 - 1])) {
            x43[x45 - 1] = '\\0';
            --x45;
        }
    }

    return x43;
}

static bool x46(char* x47, char** x48, char** x49) {

```

```

char* x50;

if (x47 == NULL || x48 == NULL || x49 == NULL) {
    return false;
}

x50 = strchr(x47, '=');
if (x50 != NULL) {
    *x50 = '\0';
    *x48 = x42(x47);
    *x49 = x42(x50 + 1);
    return **x48 != '\0';
}

x50 = x47;
while (*x50 && !isspace((unsigned char)*x50)) {
    ++x50;
}

if (*x50 == '\0') {
    *x48 = x42(x47);
    *x49 = NULL;
    return **x48 != '\0';
}

*x50 = '\0';
++x50;
*x48 = x42(x47);
*x49 = x42(x50);

return **x48 != '\0';
}

static bool x51(const char* x52, uint64_t* x53) {
    char* x54;
    unsigned long long x55;

    if (x52 == NULL || x53 == NULL || *x52 == '\0') {
        return false;
    }

    while (*x52 && isspace((unsigned char)*x52)) {
        ++x52;
    }

    if (*x52 == '-') {
        return false;
    }

    x55 = strtoull(x52, &x54, 10);
    if (x54 == x52 || *x42(x54) != '\0') {
        return false;
    }
}

```

```

    if (x55 > UINT64_MAX) {
        return false;
    }

    *x53 = (uint64_t)x55;
    return true;
}

static bool x56(char* x57, uint64_t x58, const char* x59) {
    uint64_t x60 = 0;

    if (x57 == NULL || x59 == NULL || x58 == 0) {
        return false;
    }

    while (*x59) {
        char x61 = *x59++;

        if (x61 == '\\') {
            x61 = *x59++;
            if (x61 == '\\0') {
                x61 = '\\';
                --x59;
            } else if (x61 == 'n') {
                x61 = '\n';
            } else if (x61 == 't') {
                x61 = '\t';
            } else if (x61 == 'r') {
                x61 = '\r';
            } else if (x61 == '\\\\') {
                x61 = '\\';
            }
        }

        if (x60 + 1 >= x58) {
            return false;
        }

        x57[x60++] = x61;
    }

    x57[x60] = '\\0';
    return true;
}

static void x62(x28* x63) {
    if (x63 == NULL) {
        return;
    }

    memset(x63, 0, sizeof(*x63));
    x63->x16 = true;
    (void)x32(x63->x17, sizeof(x63->x17), "x33");
    (void)x32(x63->x18, sizeof(x63->x18), "x1");
}

```

```

(void)x32(x63->x19, sizeof(x63->x19), "0123456789");
x63->x20 = 7;
(void)x32(x63->x21, sizeof(x63->x21), "x15");
x63->x22 = x11;
(void)x32(x63->x23, sizeof(x63->x23), "x31");
(void)x32(x63->x24, sizeof(x63->x24), "x33.txt");
(void)x32(x63->x25, sizeof(x63->x25), "\n");
(void)x32(x63->x26, sizeof(x63->x26), "");
(void)x32(x63->x27, sizeof(x63->x27), "");
}

static void x64(x31* x65) {
    if (x65 != NULL) {
        memset(x65, 0, sizeof(*x65));
    }
}

static bool x66(x31* x67, const x28* x68) {
    if (x67 == NULL || x68 == NULL || x67->x30 >= x2) {
        return false;
    }

    x67->x29[x67->x30++] = *x68;
    return true;
}

static bool x69(const char* x70, x15* x71) {
    if (x70 == NULL || x71 == NULL) {
        return false;
    }

    if (x39(x70, "cartesian") || x39(x70, "product") || x39(x70, "combinations"))
{
        *x71 = x11;
        return true;
    }

    if (x39(x70, "literal") || x39(x70, "echo")) {
        *x71 = x12;
        return true;
    }

    if (x39(x70, "repeat")) {
        *x71 = x13;
        return true;
    }

    if (x39(x70, "reverse")) {
        *x71 = x14;
        return true;
    }

    return false;
}

```

```
static bool x72(void* x4, const char* x6, uint64_t x7) {
    FILE* x73 = (FILE*)x4;
    return fwrite(x6, sizeof(char), (size_t)x7, x73) == x7;
}
```

```
static bool x74(void* x4, char x9) {
    FILE* x73 = (FILE*)x4;
    return fputc((unsigned char)x9, x73) != EOF;
}
```

```
static bool x75(x3* x76, const char* x77) {
    FILE* x73;

    if (x76 == NULL || x77 == NULL || *x77 == '\\0') {
        return false;
    }

    if (x39(x77, "stdout") || x39(x77, "-")) {
        *x76 = (x3){
            .x4 = stdout,
            .x5 = x72,
            .x8 = x74,
            .x10 = false
        };
        return true;
    }

    x73 = fopen(x77, "w");
    if (x73 == NULL) {
        return false;
    }

    *x76 = (x3){
        .x4 = x73,
        .x5 = x72,
        .x8 = x74,
        .x10 = true
    };

    return true;
}
```

```
static void x78(x3* x79) {
    if (x79 != NULL && x79->x4 != NULL && x79->x10) {
        fclose((FILE*)x79->x4);
        x79->x4 = NULL;
        x79->x10 = false;
    }
}
```

```
static bool x80(x3* x81, const char* x82) {
    uint64_t x83;
```

```

    if (x81 == NULL || x82 == NULL) {
        return false;
    }

    x83 = (uint64_t)strlen(x82);
    if (x83 == 0) {
        return true;
    }

    return x81->x5(x81->x4, x82, x83);
}

static bool x84(uint64_t x85, uint64_t x86, uint64_t* x87) {
    uint64_t x88;

    if (x87 == NULL || x85 == 0) {
        return false;
    }

    *x87 = 1;
    for (x88 = 0; x88 < x86; ++x88) {
        if (*x87 > UINT64_MAX / x85) {
            return false;
        }
        *x87 *= x85;
    }

    return true;
}

static bool x89(const x28* x90) {
    uint64_t x91;

    if (x90 == NULL) {
        return false;
    }

    if (x90->x17[0] == '\0' || x90->x18[0] == '\0' ||
        x90->x21[0] == '\0' || x90->x23[0] == '\0' ||
        x90->x24[0] == '\0') {
        return false;
    }

    if (x90->x20 == 0) {
        return false;
    }

    if (x90->x22 == x11) {
        if (x90->x20 > x0) {
            return false;
        }
        if (strlen(x90->x19) == 0) {
            return false;
        }
    }
}

```



```

        if (!x84((uint64_t)strlen(x90->x19), x90->x20, &x91)) {
            return false;
        }
    }

    return true;
}

static bool x92(x3* x93, const x28* x94, const char* x95, uint64_t x96) {
    if (x93 == NULL || x94 == NULL || x95 == NULL) {
        return false;
    }

    if (!x80(x93, x94->x26)) {
        return false;
    }

    if (x96 > 0 && !x93->x5(x93->x4, x95, x96)) {
        return false;
    }

    if (!x80(x93, x94->x27)) {
        return false;
    }

    if (!x80(x93, x94->x25)) {
        return false;
    }

    return true;
}

static bool x97(const x28* x98, x3* x99) {
    uint64_t x100;
    uint64_t x101;
    uint64_t x102;
    char x103[x0 + 1];

    if (x98 == NULL || x99 == NULL) {
        return false;
    }

    x100 = (uint64_t)strlen(x98->x19);
    if (!x84(x100, x98->x20, &x101)) {
        return false;
    }

    for (x102 = 0; x102 < x101; ++x102) {
        uint64_t x104 = x102;
        int64_t x105;

        for (x105 = (int64_t)x98->x20 - 1; x105 >= 0; --x105) {
            uint64_t x106 = x104 % x100;
            x103[x105] = x98->x19[x106];
        }
    }
}

```

```

        x104 /= x100;
    }

    x103[x98->x20] = '\0';

    if (!x92(x99, x98, x103, x98->x20)) {
        return false;
    }
}

return true;
}

static bool x107(const x28* x108, x3* x109) {
    if (x108 == NULL || x109 == NULL) {
        return false;
    }

    return x92(x109, x108, x108->x19, (uint64_t)strlen(x108->x19));
}

static bool x110(const x28* x111, x3* x112) {
    uint64_t x113;

    if (x111 == NULL || x112 == NULL) {
        return false;
    }

    for (x113 = 0; x113 < x111->x20; ++x113) {
        if (!x92(x112, x111, x111->x19, (uint64_t)strlen(x111->x19))) {
            return false;
        }
    }

    return true;
}

static bool x114(const x28* x115, x3* x116) {
    size_t x117;
    char x118[x1];
    size_t x119;

    if (x115 == NULL || x116 == NULL) {
        return false;
    }

    x117 = strlen(x115->x19);
    if (x117 >= sizeof(x118)) {
        return false;
    }

    for (x119 = 0; x119 < x117; ++x119) {
        x118[x119] = x115->x19[x117 - 1 - x119];
    }
}

```

```

    x118[x117] = '\\0';

    return x92(x116, x115, x118, (uint64_t)x117);
}

static bool x120(const x28* x121) {
    x3 x122;
    bool x123;

    if (!x89(x121)) {
        fprintf(stderr, "Invalid object specification: %s\\n", x121 ? x121->x17 : "
(null)");
        return false;
    }

    if (!x75(&x122, x121->x24)) {
        perror("Error opening output target");
        return false;
    }

    if (x121->x22 == x11) {
        x123 = x97(x121, &x122);
    } else if (x121->x22 == x12) {
        x123 = x107(x121, &x122);
    } else if (x121->x22 == x13) {
        x123 = x110(x121, &x122);
    } else if (x121->x22 == x14) {
        x123 = x114(x121, &x122);
    } else {
        x123 = false;
    }

    x78(&x122);
    return x123;
}

static bool x124(const x31* x125) {
    uint64_t x126;

    if (x125 == NULL || x125->x30 == 0) {
        return false;
    }

    for (x126 = 0; x126 < x125->x30; ++x126) {
        if (!x120(&x125->x29[x126])) {
            return false;
        }
    }

    return true;
}

static bool x127(x28* x128, const char* x129, const char* x130) {
    uint64_t x131;

```

```
x15 x132;

if (x128 == NULL || x129 == NULL || x130 == NULL) {
    return false;
}

if (x39(x129, "object") || x39(x129, "object_name") || x39(x129, "name")) {
    return x32(x128->x17, sizeof(x128->x17), x130);
}

if (x39(x129, "input_name")) {
    return x32(x128->x18, sizeof(x128->x18), x130);
}

if (x39(x129, "input") || x39(x129, "input_value") || x39(x129, "alphabet")) {
    return x56(x128->x19, sizeof(x128->x19), x130);
}

if (x39(x129, "input_width") || x39(x129, "width") ||
    x39(x129, "length") || x39(x129, "depth")) {
    if (!x51(x130, &x131)) {
        return false;
    }
    x128->x20 = x131;
    return true;
}

if (x39(x129, "flow_name")) {
    return x32(x128->x21, sizeof(x128->x21), x130);
}

if (x39(x129, "flow") || x39(x129, "flow_type") || x39(x129,
"processing_flow")) {
    if (!x69(x130, &x132)) {
        return false;
    }
    x128->x22 = x132;
    return true;
}

if (x39(x129, "output_name")) {
    return x32(x128->x23, sizeof(x128->x23), x130);
}

if (x39(x129, "output") || x39(x129, "output_target") || x39(x129, "file")) {
    return x32(x128->x24, sizeof(x128->x24), x130);
}

if (x39(x129, "separator") || x39(x129, "sep")) {
    return x56(x128->x25, sizeof(x128->x25), x130);
}

if (x39(x129, "prefix")) {
    return x56(x128->x26, sizeof(x128->x26), x130);
}
```

```

}

if (x39(x129, "suffix")) {
    return x56(x128->x27, sizeof(x128->x27), x130);
}

return false;
}

static bool x133(const char* x134, x31* x135) {
    FILE* x136;
    char x137[x200];
    uint64_t x138 = 0;
    x28 x139;
    bool x140 = false;

    if (x134 == NULL || x135 == NULL) {
        return false;
    }

    x136 = fopen(x134, "r");
    if (x136 == NULL) {
        return false;
    }

    x64(x135);
    x62(&x139);

    while (fgets(x137, sizeof(x137), x136) != NULL) {
        char* x141;
        char* x142;
        char* x143;

        ++x138;
        x141 = x42(x137);

        if (*x141 == '\0') {
            continue;
        }

        if (x39(x141, "end")) {
            if (!x66(x135, &x139)) {
                fclose(x136);
                return false;
            }
            x62(&x139);
            x140 = false;
            continue;
        }

        if (!x46(x141, &x142, &x143) || x143 == NULL) {
            fprintf(stderr, "Invalid config line %llu\n", (unsigned long
long)x138);
            fclose(x136);

```

```

        return false;
    }

    if (x39(x142, "object") || x39(x142, "object_name")) {
        if (x140) {
            if (!x66(x135, &x139)) {
                fclose(x136);
                return false;
            }
            x62(&x139);
        }
        x140 = true;
    } else {
        x140 = true;
    }

    if (!x127(&x139, x142, x143)) {
        fprintf(stderr, "Invalid config key or value on line %llu: %s\n",
            (unsigned long long)x138, x142);
        fclose(x136);
        return false;
    }
}

if (x140) {
    if (!x66(x135, &x139)) {
        fclose(x136);
        return false;
    }
}

fclose(x136);
return x135->x30 > 0;
}

static bool x144(const char* x145, char* x146, uint64_t x147, const char* x148) {
    char x149[x200];
    size_t x150;

    if (x145 == NULL || x146 == NULL || x147 == 0 || x148 == NULL) {
        return false;
    }

    printf("%s [%s]: ", x145, x148);
    fflush(stdout);

    if (fgets(x149, sizeof(x149), stdin) == NULL) {
        return false;
    }

    x150 = strlen(x149);
    if (x150 > 0 && x149[x150 - 1] == '\n') {
        x149[x150 - 1] = '\0';
    }
}

```

```

    if (x149[0] == '\0') {
        return x32(x146, x147, x148);
    }

    return x32(x146, x147, x149);
}

static bool x151(const char* x152, uint64_t* x153, uint64_t x154) {
    char x155[x0];
    char x156[x0];

    if (x152 == NULL || x153 == NULL) {
        return false;
    }

    snprintf(x156, sizeof(x156), "%llu", (unsigned long long)x154);

    if (!x144(x152, x155, sizeof(x155), x156)) {
        return false;
    }

    return x51(x155, x153);
}

static bool x157(x31* x158) {
    x28 x159;
    char x160[x0];
    bool x161 = true;

    if (x158 == NULL) {
        return false;
    }

    x64(x158);

    while (x161 && x158->x30 < x2) {
        x62(&x159);

        if (!x144("object_name", x159.x17, sizeof(x159.x17), x159.x17)) {
            return false;
        }
        if (!x144("input_name", x159.x18, sizeof(x159.x18), x159.x18)) {
            return false;
        }
        if (!x144("input_value", x159.x19, sizeof(x159.x19), x159.x19)) {
            return false;
        }
        if (!x151("input_width", &x159.x20, x159.x20)) {
            return false;
        }
        if (!x144("flow_name", x159.x21, sizeof(x159.x21), x159.x21)) {
            return false;
        }
    }
}

```

```

        if (!x144("flow_type cartesian|literal|repeat|reverse", x160,
sizeof(x160), "cartesian")) {
            return false;
        }
        if (!x69(x160, &x159.x22)) {
            fprintf(stderr, "Unknown flow_type: %s\n", x160);
            return false;
        }
        if (!x144("output_name", x159.x23, sizeof(x159.x23), x159.x23)) {
            return false;
        }
        if (!x144("output_target file|stdout|-", x159.x24, sizeof(x159.x24),
x159.x24)) {
            return false;
        }
        if (!x144("separator", x160, sizeof(x160), "\\n")) {
            return false;
        }
        if (!x56(x159.x25, sizeof(x159.x25), x160)) {
            return false;
        }
        if (!x144("prefix", x160, sizeof(x160), "")) {
            return false;
        }
        if (!x56(x159.x26, sizeof(x159.x26), x160)) {
            return false;
        }
        if (!x144("suffix", x160, sizeof(x160), "")) {
            return false;
        }
        if (!x56(x159.x27, sizeof(x159.x27), x160)) {
            return false;
        }

        if (!x66(x158, &x159)) {
            return false;
        }

        if (!x144("add another object? y|n", x160, sizeof(x160), "n")) {
            return false;
        }

        x161 = x39(x160, "y") || x39(x160, "yes");
    }

    return x158->x30 > 0;
}

static bool x162(const char* x163) {
    FILE* x164;

    if (x163 == NULL) {
        return false;
    }

```



```

x164 = fopen(x163, "w");
if (x164 == NULL) {
    return false;
}

fprintf(x164,
    "# configurator_v2 fabric config\n"
    "object_name=x33\n"
    "input_name=x1\n"
    "input_value=0123456789\n"
    "input_width=3\n"
    "flow_name=x15\n"
    "flow_type=cartesian\n"
    "output_name=x31\n"
    "output_target=x33.txt\n"
    "separator=\\n\n"
    "prefix=\n"
    "suffix=\n"
    "end\n"
    "\n"
    "object_name=x34\n"
    "input_name=x35\n"
    "input_value=emerge\n"
    "input_width=1\n"
    "flow_name=x36\n"
    "flow_type=reverse\n"
    "output_name=x37\n"
    "output_target=stdout\n"
    "separator=\\n\n"
    "end\n");

fclose(x164);
return true;
}

static void x165(const char* x166) {
    fprintf(stderr,
        "Usage:\n"
        " %s --ui\n"
        " %s --config <path>\n"
        " %s --sample-config <path>\n"
        " %s [object flags]\n"
        "\n"
        "Object flags:\n"
        "  --object-name <name>      runtime object name\n"
        "  --input-name <name>       runtime input name\n"
        "  --input-value <value>     object input payload/alphabet\n"
        "  --input-width <n>         width/count/depth\n"
        "  --flow-name <name>        runtime processing-flow name\n"
        "  --flow-type <type>        cartesian|literal|repeat|reverse\n"
        "  --output-name <name>      runtime output name\n"
        "  --output-target <target>  file path, stdout, or -\n"
        "  --separator <text>        supports \\n, \\t, \\r,

```

```

\\\\\\\\\\\\\\n"
    "  --prefix <text>                supports escapes\n"
    "  --suffix <text>                supports escapes\n"
    "\n"
    "Aliases:\n"
    "  -n <name>  -i <value>  -w <n>  -f <type>  -o <target>\n",
    x166, x166, x166, x166);
}

static int x167(int x168, char** x169, x31* x170, bool* x171) {
    int x172;
    x28 x173;
    bool x174 = false;

    if (x170 == NULL || x171 == NULL) {
        return 1;
    }

    *x171 = false;
    x64(x170);

    if (x168 <= 1) {
        x165(x169[0]);
        return 0;
    }

    for (x172 = 1; x172 < x168; ++x172) {
        if (x39(x169[x172], "--help") || x39(x169[x172], "-h")) {
            x165(x169[0]);
            return 0;
        }

        if (x39(x169[x172], "--ui")) {
            if (!x157(x170)) {
                return 1;
            }
            *x171 = true;
            return 0;
        }

        if (x39(x169[x172], "--config")) {
            if (x172 + 1 >= x168) {
                x165(x169[0]);
                return 1;
            }
            if (!x133(x169[x172 + 1], x170)) {
                perror("Error loading config");
                return 1;
            }
            *x171 = true;
            return 0;
        }

        if (x39(x169[x172], "--sample-config")) {

```

```

        if (x172 + 1 >= x168) {
            x165(x169[0]);
            return 1;
        }
        if (!x162(x169[x172 + 1])) {
            perror("Error writing sample config");
            return 1;
        }
        *x171 = false;
        return 0;
    }
}

x62(&x173);

for (x172 = 1; x172 < x168; ++x172) {
    const char* x175 = x169[x172];
    const char* x176 = NULL;

    if (x172 + 1 < x168) {
        x176 = x169[x172 + 1];
    }

    if (x176 == NULL) {
        x165(x169[0]);
        return 1;
    }

    if (x39(x175, "--object-name") || x39(x175, "-n")) {
        if (!x127(&x173, "object_name", x176)) {
            return 1;
        }
    } else if (x39(x175, "--input-name")) {
        if (!x127(&x173, "input_name", x176)) {
            return 1;
        }
    } else if (x39(x175, "--input-value") || x39(x175, "-i")) {
        if (!x127(&x173, "input_value", x176)) {
            return 1;
        }
    } else if (x39(x175, "--input-width") || x39(x175, "-w")) {
        if (!x127(&x173, "input_width", x176)) {
            return 1;
        }
    } else if (x39(x175, "--flow-name")) {
        if (!x127(&x173, "flow_name", x176)) {
            return 1;
        }
    } else if (x39(x175, "--flow-type") || x39(x175, "-f")) {
        if (!x127(&x173, "flow_type", x176)) {
            return 1;
        }
    } else if (x39(x175, "--output-name")) {
        if (!x127(&x173, "output_name", x176)) {

```

```

        return 1;
    }
} else if (x39(x175, "--output-target") || x39(x175, "-o")) {
    if (!x127(&x173, "output_target", x176)) {
        return 1;
    }
} else if (x39(x175, "--separator")) {
    if (!x127(&x173, "separator", x176)) {
        return 1;
    }
} else if (x39(x175, "--prefix")) {
    if (!x127(&x173, "prefix", x176)) {
        return 1;
    }
} else if (x39(x175, "--suffix")) {
    if (!x127(&x173, "suffix", x176)) {
        return 1;
    }
} else {
    fprintf(stderr, "Unknown flag: %s\n", x175);
    x165(x169[0]);
    return 1;
}

++x172;
x174 = true;
}

if (!x174) {
    x165(x169[0]);
    return 0;
}

if (!x66(x170, &x173)) {
    return 1;
}

*x171 = true;
return 0;
}

int main(int x177, char** x178) {
    x31 x179;
    bool x180;
    int x181;

    x181 = x167(x177, x178, &x179, &x180);
    if (x181 != 0) {
        return x181;
    }

    if (!x180) {
        return 0;
    }
}

```

```
    if (!x124(&x179)) {  
        return 1;  
    }  
  
    return 0;  
}
```